ATLAS Backend

What every part does, with real examples

This report explains the backend of the ATLAS repository: the FastAPI service in `apps/api` and the small Node web server in `apps/web-server` that sits in front of it. It covers both product areas, **Conversation Quality Analytics (CQA)** and the **Data Studio**, plus the shared foundation, storage, security and deployment. Each part comes with an example request and the response the API actually returned.

Item	Value
Code reviewed	Commit `3c6abb7` on branch `deploy/cicd` (working tree clean)
Backend size	≈ 9,400 lines of Python in `apps/api/app`, 74 HTTP endpoints, 300-line Node proxy
Tests	190 tests in `apps/api/tests`: **190 passed** on 7 November 2026
How examples were made	The real app was started in-process (`create_app`) against an empty, throwaway workspace, using the synthetic fixtures shipped in the repo (group `FIXTURE`, dataset `customers_churn.csv`) and the development mock evaluator. IDs and timestamps will differ on your machine.
Important	The mock evaluator is **not AI**, and the development checklist is **not the official checklist**. Their scores show how the system works, not real quality results.

Contents

1. Overview . . . 3
1.1 Architecture . . . 3
1.2 Where things live . . . 3
1.3 Life of a request . . . 4

2. Foundation: app, settings, errors, security, storage . . . 5
2.1 App factory and lifespan . . . 5
2.2 Configuration . . . 5
2.3 Errors . . . 6
2.4 Security guards . . . 6
2.5 Storage: the workspace . . . 7

3. CQA: Conversation Quality Analytics . . . 8
3.1 Importing conversations . . . 9
3.2 Browsing conversations . . . 10
3.3 Call recordings . . . 12
3.4 Checklists and versions . . . 12
3.5 Running evaluations . . . 14
3.6 The evaluator contract (cqa-eval/1) . . . 15
3.7 Evaluator adapters . . . 18
3.8 The scoring engine . . . 18
3.9 Appeals and reviewer overrides . . . 20
3.10 Overview and audit . . . 21

4. Data Studio . . . 23
4.1 Datasets: upload, import, replace . . . 23
4.2 Profiling . . . 24
4.3 Query, aggregate, export . . . 25
4.4 Charts . . . 26
4.5 Preparation recipes (transforms) . . . 26
4.6 Models and experiments . . . 27
4.7 Analysis plug-ins . . . 30
4.8 Findings, pitch deck and saved views . . . 31

5. The web server (apps/web-server) . . . 31

6. Running, deployment and tests . . . 32
6.1 Local development . . . 32
6.2 Docker and Compose . . . 32
6.3 Tests . . . 32
6.4 Limits and things to know . . . 32

Appendix: endpoint reference . . . 33

1. Overview

The backend is one Python process (FastAPI, Pydantic v2, pandas, scikit-learn) that stores everything as files on local disk; there is no database. It serves two product areas that share the same foundation:

- **CQA — Conversation Quality Analytics** (`app/cqa`). Imports call-centre conversations (chats and voice calls in Russian, Kazakh and English) from the hackathon organizers' API, evaluates each one against a configurable, versioned checklist through a pluggable **evaluator** (the team's AI model plugs in here), computes every score with a deterministic engine, and handles appeals with a full audit trail. This is the part the deployed product exposes.
- **Data Studio** (`app/api`, `app/services`, `app/extensions`, `app/challenge`). A general, local-first data tool: upload CSV, Excel or Parquet files, profile them, filter and chart them, build preparation recipes, train baseline machine-learning models, run statistical analyses, and collect findings for a pitch deck. By default the public web server does not forward these endpoints.

1.1 Architecture

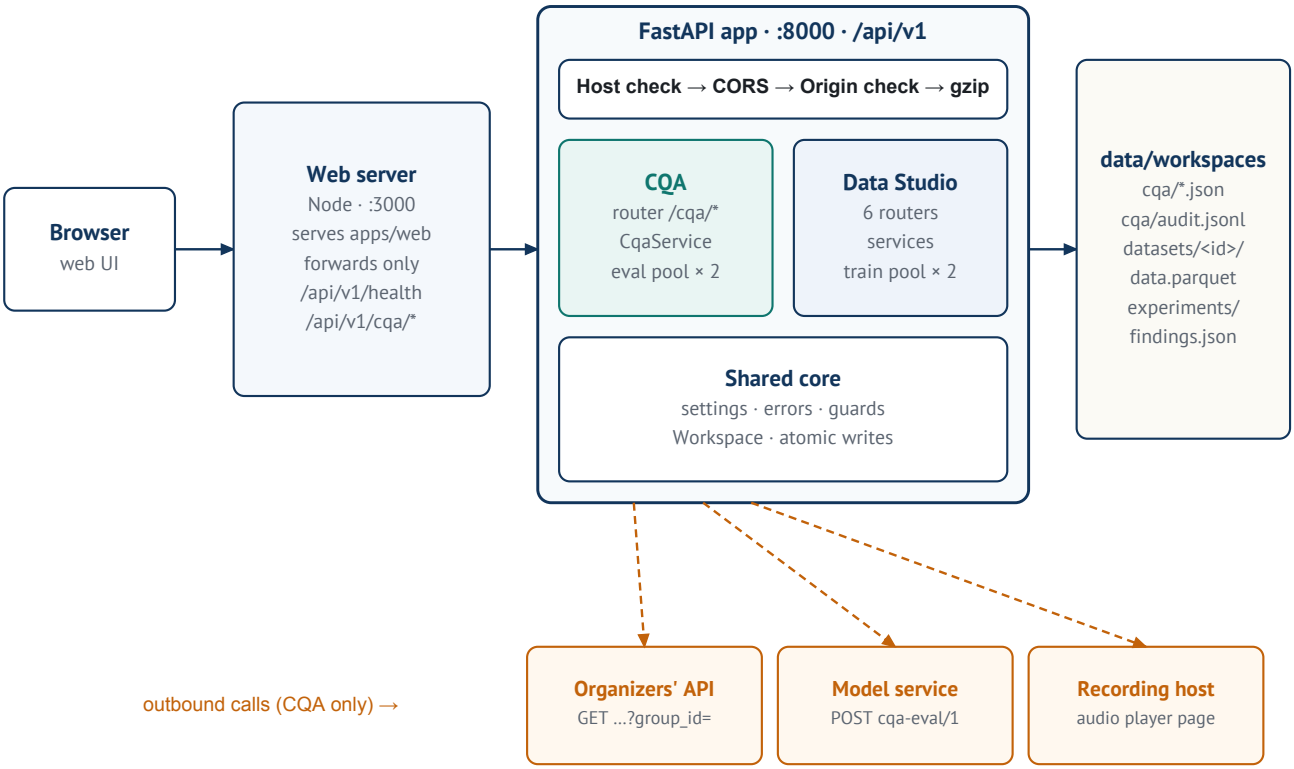


Figure 1. Request path and dependencies. Solid arrows are normal request flow; dashed arrows are the only outbound network calls the backend makes, all from CQA.

A request from the browser goes to the Node web server on port 3000. Static files are served directly. Paths under `/api/` are checked against an allow-list and forwarded to FastAPI on port 8000. Inside FastAPI, middleware checks the request, a thin route validates input, a service does the work, and the result is written to `data/workspaces`.

1.2 Where things live

Path	Responsibility
`app/main.py`	App factory: middleware order, error handlers, routers, and the lifespan that creates the long-lived services.
`app/core/`	`config.py` settings (every value overridable with an `ATLAS_` env var), `errors.py` error envelope, `security.py` origin guard.

Path	Responsibility
`app/storage/`	Workspace on disk, safe IDs and paths, atomic JSON writes, an in-memory DataFrame cache.
`app/api/deps.py`	Dependency injection for routes, CSV streaming, formula-safe exports, safe download file names.
`app/api/routes/`	Data Studio routes: system, datasets, analysis, analyses, experiments (models), findings.
`app/services/`	Data Studio logic: datasets, file readers, profiler, transforms, query and charts, modeling, reporting, shared filters.
`app/schemas/`	Pydantic request and response models for the Data Studio.
`app/extensions/`	Analysis plug-in registry and six built-in analyses.
`app/challenge/`	Place for case-specific analyses and endpoints (currently a template).
`app/cqa/`	Everything CQA: router, service, store, ingestion, checklists, evaluations, contract, adapters, scoring, appeals, audio, schemas, fixtures.
`app/utils/frames.py`	DataFrame helpers: column kind inference, JSON-safe rows, sampling.
`apps/web-server/server.mjs`	Dependency-free Node server: static files and the `/api` proxy.
`apps/api/tests/`	15 test modules, 190 tests.

1.3 Life of a request

Middleware runs in the reverse of the order it was added, so every request passes through these layers:

- **Trusted host** rejects any `Host` header outside `ATLAS_ALLOWED_HOSTS` (blocks DNS rebinding).
- **CORS** lets only the configured web origins read responses.
- **Origin guard** refuses state-changing requests sent by a page from another origin.
- **GZip** compresses responses over 2 KB.
- **Route** validates path, query and body with Pydantic, calls one service method and returns a typed model.
- **Service** does the work and persists through the store or workspace. Any error becomes the JSON envelope `{"error": {code, message, details}}`.

> **Note.** Routes stay thin on purpose: they validate input and pick status codes. All logic lives in services, so you can test it without HTTP. The examples in this report were produced the same way the tests work: `create_app(Settings(data_dir=<temp dir>))`.

2. Foundation: app, settings, errors, security, storage

2.1 App factory and lifespan

`create_app(settings)` builds the FastAPI app. Its lifespan hook runs once at start-up and creates the long-lived objects on `app.state`:

Object	What it owns
`Workspace`	All paths under `ATLAS_DATA_DIR` and an LRU cache of up to 4 loaded tables.
`DatasetService`	Dataset upload, import, preview, profile, samples, derived datasets.
`ExperimentManager`	A 2-thread pool that trains models in the background.
`ReportStore`	Findings, the pitch deck and saved views (JSON documents).
`CqaService`	The CQA store plus checklist, evaluation (2-thread pool), appeal and audio services.

On shutdown the two thread pools stop and queued work is cancelled. What happens to that work on the next start is covered in sections 3.5 and 4.6. Routes reach these objects through small dependency functions, for example `DatasetsDep` or `Cqa`.

2.2 Configuration

`Settings` (pydantic-settings) reads every field from an environment variable with the `ATLAS_` prefix, or from a `.env` file in the working directory. List values are written as JSON. The ones that matter most:

Variable	Default	Meaning
`ATLAS_DATA_DIR`	`data/workspaces`	Where everything is stored (`/data` in Docker).
`ATLAS_CORS_ORIGINS`	localhost:3000, 127.0.0.1:3000	Web origins allowed to call the API from a browser.
`ATLAS_ALLOWED_HOSTS`	127.0.0.1, localhost	Host names the API answers to.
`ATLAS_MAX_UPLOAD_MB`	500	Upload size limit.
`ATLAS_MAX_ROWS` / `_COLUMNS`	5,000,000 / 2,000	Table size limits at import.
`ATLAS_MODEL_MAX_TRAIN_ROWS`	200,000	Training rows above this are sampled.
`ATLAS_CQA_SOURCE_URL`	none	Organizers' GET link. Contains a signed token, so it lives only in `.env`.
`ATLAS_CQA_FIXTURES_ENABLED`	true	Allow importing the synthetic `FIXTURE...` groups.
`ATLAS_CQA_EVALUATOR`	`mock`	Default evaluator: `mock`, `http`, or a registered adapter name.
`ATLAS_CQA_EVALUATOR_URL`	none	Model service endpoint for the `http` adapter.
`ATLAS_CQA_EVALUATOR_ALLOW_REMOTE`	false	Must be true to send transcripts to a non-local host.
`ATLAS_CQA_EVALUATOR_TIMEOUT_S`	180	Time limit for one model call.
`ATLAS_CQA_EVALUATION_WORKERS`	2	Evaluations that run in parallel.
`ATLAS_CQA_AUDIO_HOSTS`	powerplatform.com, dynamics.com	Hosts the API may contact to resolve call recordings.

Example `.env`

```
# .env (never committed): connect the real data source and a local model service
ATLAS_CQA_SOURCE_URL=https://<organizer-host>/...?token=<signed>
ATLAS_CQA_EVALUATOR=http
ATLAS_CQA_EVALUATOR_URL=http://127.0.0.1:9000/evaluate
ATLAS_CORS_ORIGINS=["http://localhost:3000"]
```

2.3 Errors

Every failure reaches the client in one shape, so the UI can show `message` and branch on `code`. Services raise typed errors; handlers in `core/errors.py` convert them.

Raised as	HTTP	Default code	Typical use
`AppError`	400	`bad_request`	Base class for expected failures.
`NotFoundError`	404	`not_found`	Unknown dataset, conversation, checklist, appeal…
`ValidationFailed`	422	`validation_failed`	Business-rule violations (bad option, wrong column type).
`ConflictError`	409	`conflict`	Stale checklist edit, duplicate appeal, checklist in use.
`PayloadTooLarge`	413	`payload_too_large`	Upload over the size limit, zip bomb.
`SourceError` (CQA)	502	`source_unavailable`	Organizers' API unreachable or returned bad data.
Pydantic validation	422	`invalid_request`	Wrong field types or patterns; `details` lists each field.
Anything else	500	`internal_error`	Logged with stack trace; the client gets a generic message.

GET /api/v1/cqa/conversations/does-not-exist

← 404 Not Found


```
{
  "error": {
    "code": "conversation_not_found",
    "message": "Conversation does-not-exist does not exist.",
    "details": null
  }
}
```


POST /api/v1/cqa/imports


```
{"group_id": "bad id!"}
```


← 422 Unprocessable Content


```
{
  "error": {
    "code": "invalid_request",
    "message": "Request validation failed.",
    "details": [
      {
        "loc": ["body", "group_id"],
        "msg": "String should match pattern '^[A-Za-z0-9][A-Za-z0-9._-]{0,63}$'"
      }
    ]
  }
}
```


2.4 Security guards

The API has **no login**; that is a deliberate non-goal for a tool that runs on localhost. Instead it defends against the realistic attacker, a malicious web page open in the same browser, and it never turns user input into file paths or URLs without checks.

- **DNS rebinding.** A hostile domain can be re-pointed at 127.0.0.1. Only `Host` values in `ATLAS_ALLOWED_HOSTS` are accepted. Real example: the test client's default `Host: testserver` was rejected with `400 Invalid host header` while these examples were being captured.
- **Cross-site writes.** CORS stops a foreign page from reading responses, but not from sending a form POST. `OriginGuard` refuses POST, PUT, PATCH and DELETE requests whose `Origin` is neither an allowed web origin nor the API itself. Requests without `Origin` (curl, scripts, tests) cannot come from a web page and are allowed.
- **Paths.** Every ID must be 32 lowercase hex characters, file names are sanitized, and `safe_child()` refuses any path that would leave the workspace.

- **Outbound calls.** The organizers' URL comes only from `.env`, recordings are fetched only from allow-listed https hosts, and transcripts go only to a loopback model service unless remote sending is explicitly allowed.

POST /api/v1/cqa/checklists/dev-sample (Origin: https://evil.example)

← 403 Forbidden

```
{
  "error": {
    "code": "forbidden_origin",
    "message": "Requests from https://evil.example may not change this workspace. Add the web app's
        origin to ATLAS_CORS_ORIGINS if this is intended.",
    "details": null
  }
}
```

Formula-safe CSV exports

Spreadsheet programs execute cells that start with `=`, `+`, `-`, `@`, tab or CR. Every CSV export prefixes such text cells with `'`, but leaves real numbers such as `-5` alone. Exact values remain available as Parquet or as the original upload. Output of `neutralize_formulas()`:

Cell text	Exported as
`=HYPERLINK("http://x")`	`'=HYPERLINK("http://x")`
`-5`	`-5`
`+7 701 000 00 00`	`'+7 701 000 00 00`
`@SUM(A1)`	`'@SUM(A1)`
`Alice`	`Alice`

Download file names

HTTP headers are Latin-1, so a Cyrillic file name cannot be sent as-is. `attachment()` sends an ASCII fallback plus the real UTF-8 name (RFC 6266 / 5987). For `Клиенты · filtered.csv`:

Content-Disposition

```
attachment; filename="filtered.csv";
    filename*=UTF-8''%D0%9A%D0%BB%D0%B8%D0%B5%D0%BD%D1%82%D1%8B%20%C2%B7%20filtered.csv
```

2.5 Storage: the workspace

`Workspace` owns every path the Data Studio reads or writes; `CqaStore` does the same for CQA under `cqa/`. Writes are atomic: content goes to a temp file in the same folder, then `os.replace()` swaps it in, so a crash never leaves a half-written file.

Layout under ATLAS_DATA_DIR

```
data/workspaces/
  datasets/<id>/manifest.json         metadata: name, status, format, columns, lineage
  datasets/<id>/original.csv          the untouched upload
  datasets/<id>/data.parquet          canonical parsed table used by every service
  datasets/<id>/profile.json          cached profile
  datasets/<id>/previous/             old version kept while a replacement is pending
  experiments/<id>.json               experiment record (metrics, importances, warnings)
  experiments/<id>.predictions.parquet
  findings.json  pitch.json  views.json
  cqa/conversations/<id>.<hash8>.json one file per conversation
  cqa/checklists/<id>.json            a checklist with its full version history
  cqa/evaluations/<id>.json           one evaluation (checklist snapshot, results, overrides)
  cqa/appeals/<id>.json               one appeal and its decision
  cqa/imports.json                    the last 200 import runs
  cqa/audit.jsonl                     append-only audit trail
```

- Loaded tables are cached by `(dataset id, file modification time)`, so a re-import automatically invalidates the cache. The cache holds 4 tables by default (`ATLAS_DATAFRAME_CACHE_SIZE`).
- CQA loads all its JSON files into memory at start-up and writes through on every change. A corrupt file is skipped with a log line, never silently rewritten.
- Conversation files carry an 8-character hash suffix so IDs that differ only by case do not collide on case-insensitive file systems (macOS, Windows).

3. CQA: Conversation Quality Analytics

CQA answers one question for every conversation: **how well did the operator follow the checklist, and why?** The design separates three jobs that are often mixed up:

- the **evaluator** (AI model or mock) only gives answers: yes/no, a scale point, or an option, each with an explanation and evidence from the transcript;
- the **contract** checks those answers strictly before anything is saved;
- the **scoring engine** turns answers into points. It is the only place a score is computed, so the same answers always give the same score.

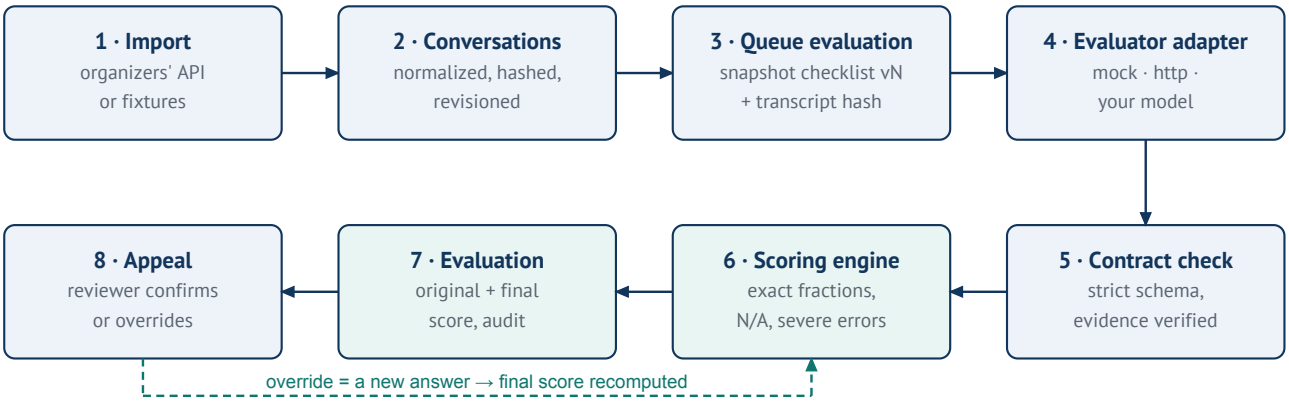


Figure 2. The CQA pipeline. Steps 1–7 run for every evaluation; step 8 changes the final score only through new answers, never by editing numbers.

Module	Job
`ingestion.py`	Fetch a group from the organizers' API or the fixtures, parse and normalize each record.
`checklists.py`	Create, edit, version, duplicate, import and export checklists.
`evaluations.py`	Queue and run evaluations in a thread pool; track pending → running → completed | failed.
`contract.py`	The evaluator contract `cqa-eval/1`: request builder and strict output validation.
`adapters/`	Evaluator plug-ins: `mock` (development only), `http` (model service), and yours.
`scoring.py`	Deterministic scoring with exact arithmetic, N/A redistribution and severe errors.
`appeals.py`	Disputes on one criterion, reviewer decisions, final-score recalculation.
`audio.py`	Turn a call's player-page link into a playable audio URL, safely.
`service.py`	`CqaService`: wires the parts together; conversation list, detail and overview.
`store.py`	JSON persistence and the audit trail.
`router.py`	29 endpoints under `/api/v1/cqa`.

3.1 Importing conversations

`POST /cqa/imports` with a `group_id` fetches that group **server-side**, so the browser never deals with CORS or sees the signed URL. With `source: "fixture"` it loads the synthetic groups shipped in `cqa/fixtures/conversations.json` (`FIXTURE`, `FIXTURE-CONTROL`) instead. The organizers' payload looks like this:

Organizers' payload format


```
{"error": "", "isSuccess": true, "data": {
  "dataset_group_id": "DEV", "total_records": 65,
  "records": [{"conversation_id": "p1-ru-001", "channel": "agent chat", "language": "RU",
               "transcript": [{"timestamp": "00:02", "speaker": "operator", "text": "..."}],
               "audio_url": null}]}}
```


`source_url_for()` takes the configured URL and replaces only its `group_id` parameter; the rest of the query (the signed token) is kept byte for byte. Error messages never include the URL. Each record is parsed on its own: a bad record is rejected with a reason and the rest of the import continues. Nothing is invented. Unknown values are kept as received and reported as warnings.

Normalization rules

Field	Accepted input → stored value	If unknown
channel	`agent chat`, `chat` → `chat`; `voice call`, `voice`, `call` → `voice`	`voice` if there is an audio URL, else `chat`; warning
language	`RU`/`RUS`/`RU-RU` → RU; `KZ`/`KK`/`KAZ`/`KK-KZ` → KZ; `EN`/`ENG`/`EN-US`/`EN-GB` → EN	kept upper-case; warning
speaker	client/customer/клиент, operator/agent/оператор, bot/бот, system/система	`unknown`; warning
timestamp	`MM:SS` or `H:MM:SS` → seconds; ISO 8601 (incl. `Z`, 7 fraction digits) → seconds since first line	kept as text; warning
transcript	a list of lines; a JSON string is decoded with a warning	record rejected
message id	assigned by position: `m1`, `m2`, … (evidence must use these)	—

Each conversation gets a **transcript hash** (SHA-256 of every line's speaker, timestamp and text). Re-importing compares hashes:

- new ID → **created**;
- same source and same hash, channel and language → **unchanged** (the group is added to `group_ids`);
- different transcript → **updated**, `revision` + 1, warning `transcript_changed`. Earlier evaluations become **stale**, and a queued evaluation fails with `transcript_changed` instead of scoring the wrong text;
- same ID from a different source (fixture vs. API) → rejected with `source_conflict`, never overwritten.

Run status is `completed` (no rejections), `partial` (some rejected) or `failed` (nothing accepted). Limits: 3,000 lines per transcript, 20,000 characters per line, 64 MB per response.

First import of the fixture group: six new conversations, one warning (a voice call without a recording).

POST /api/v1/cqa/imports


```
{"group_id": "FIXTURE", "source": "fixture"}
```


← 201 Created


```
{
  "id": "f946d4137a36462ca1cbca1fcdeadebe",
  "group_id": "FIXTURE",
  "source": "fixture",
  "status": "completed",
  "started_at": "2026-11-07T13:30:22.771617+00:00",
```

```
  "finished_at": "2026-11-07T13:30:22.773224+00:00",
  "reported_total": 6,
  "received": 6,
  "created": 6,
  "updated": 0,
  "unchanged": 0,
  "rejected": 0,
  "conversation_ids": ["fx-ru-001", "fx-kz-001", "fx-en-001", "fx-ru-002", "fx-en-002", "fx-kz-002"],
  "issues": [
    {
      "severity": "warning",
      "code": "no_recording",
      "message": "Voice call without audio_url: there is no recording to play.",
      "record_index": 3,
      "conversation_id": "fx-ru-002"
    }
  ],
  "error": null
}
```


Importing the same group again changes nothing: `created: 0`, `unchanged: 6`, `updated: 0`.

Importing a real group before `ATLAS_CQA_SOURCE_URL` is configured:

POST /api/v1/cqa/imports

```
{"group_id": "DEV"}
```

← 422 Unprocessable Content


```
{
  "error": {
    "code": "source_not_configured",
    "message": "The organizers' API is not configured. Put its GET link in .env as ATLAS_CQA_SOURCE_URL
        and restart the API.",
    "details": null
  }
}
```


3.2 Browsing conversations

`GET /cqa/conversations` filters, sorts and pages conversations in memory. Filters: free-text `q` (searches IDs and message text, case-insensitive), `language`, `channel`, `status` (`not_evaluated`, `pending`, `running`, `completed`, `failed`, `appealed`), `group_id`, `source`, `min_score`/`max_score`, and `checklist_id` (which checklist's latest evaluation counts). Sort keys: `id`, `imported_at`, `score`, `messages`, `duration`, `language`, `channel`, `evaluated_at`. The response also carries **facets** (value counts) so the UI can build its filter menus.

Russian conversations, first page (one item shown):

GET /api/v1/cqa/conversations?language=RU&page_size=2

← 200 OK


```
{
  "items": [
    {
      "id": "fx-ru-001",
      "group_ids": ["FIXTURE"],
      "channel": "chat",
      "channel_raw": "agent chat",
      "language": "RU",
      "source": "fixture",
      "has_audio": false,
      "message_count": 10,
      "speaker_counts": {"client": 4, "bot": 1, "system": 1, "operator": 4},
      "duration_seconds": 104.0,
      "preview": "Здравствуйте, как вывести деньги со счёта на карту?",
      "imported_at": "2026-11-07T13:30:22.771617+00:00",
      "updated_at": "2026-11-07T13:30:22.771617+00:00",
      "revision": 1,
      "latest_evaluation": null,
```

```
      "open_appeals": 0
    }
  ],
  "total": 2,
  "page": 1,
  "page_size": 2,
  "facets": {
    "language": [
      {"value": "EN", "count": 2},
      {"value": "KZ", "count": 2},
      {"value": "RU", "count": 2}
    ],
    "channel": [{"value": "chat", "count": 5}, {"value": "voice", "count": 1}],
    "group": [{"value": "FIXTURE", "count": 6}],
    "source": [{"value": "fixture", "count": 6}],
    "status": [{"value": "not_evaluated", "count": 6}]
  }
}
```


`GET /cqa/conversations/{id}` adds the full message list, the transcript hash, import warnings and every evaluation of that conversation. `preview` is the first client message, and `duration_seconds` is the span between the first and last timestamps.

GET /api/v1/cqa/conversations/fx-ru-001

← 200 OK


```
{
  "id": "fx-ru-001",
  "group_ids": ["FIXTURE"],
  "channel": "chat",
  "channel_raw": "agent chat",
  "language": "RU",
  "source": "fixture",
  "has_audio": false,
  "message_count": 10,
  "speaker_counts": {"client": 4, "bot": 1, "system": 1, "operator": 4},
  "duration_seconds": 104.0,
  "preview": "Здравствуйте, как вывести деньги со счёта на карту?",
  "imported_at": "2026-11-07T13:30:22.771617+00:00",
  "updated_at": "2026-11-07T13:30:22.771617+00:00",
  "revision": 1,
  "latest_evaluation": null,
  "open_appeals": 0,
  "audio_url": null,
  "messages": [
    {
      "id": "m1",
      "index": 0,
      "speaker": "client",
      "speaker_raw": "client",
      "text": "Здравствуйте, как вывести деньги со счёта на карту?",
      "timestamp": "00:00",
      "offset_seconds": 0.0,
      "at": null
    },
    {
      "id": "m2",
      "index": 1,
      "speaker": "bot",
      "speaker_raw": "bot",
      "text": "Здравствуйте! Я виртуальный помощник. Подскажу по выводу средств или соединю с
          оператором.",
      "timestamp": "00:02",
      "offset_seconds": 2.0,
      "at": null
    },
    "… 8 more messages"
  ],
  "transcript_hash": "07ed3e4c080f68dae2b3da5e6367cb2f2d0f69bc91163b433b430b027122bce5",
  "warnings": [],
  "evaluations": []
}
```

3.3 Call recordings

A call's `audio_url` points to an organizer-hosted **player page** (HTML), not to an audio file. The page contains a signed MP3 link that expires after about an hour. `GET /cqa/conversations/{id}/audio` fetches the page, extracts the `<source src>` or `<audio src>` link and returns it, so the browser can play the file directly. Safeguards:

- only **https** hosts that match `ATLAS_CQA_AUDIO_HOSTS` are contacted, both for the page and for the audio link inside it, so imported data cannot make the API request arbitrary URLs;
- at most 1 MB of the page is read; audio bodies are not downloaded;
- results are cached until 2 minutes before the signed link expires, at most 10 minutes;
- on any failure the status is `player_only` and the UI opens the original player page instead.

status	Meaning
`ready`	`stream_url` is directly playable.
`player_only`	Use `player_url` (host not allowed, page unreachable, no audio tag found).
`unavailable`	The conversation has no recording (all chats).

GET /api/v1/cqa/conversations/fx-ru-001/audio

← 200 OK


```
{
  "status": "unavailable",
  "stream_url": null,
  "player_url": null,
  "content_type": null,
  "message": "This conversation has no recording.",
  "offset_basis": "Offsets are measured from the first transcript line; the recording may start earlier."
}
```


3.4 Checklists and versions

A checklist has **sections** and up to 60 **criteria**. Each criterion has a stable `id`, a `weight`, an answer type and optional N/A and severe-error rules:

Answer type	Evaluator answers with	Normalized to 0–1 as
`binary`	`true` / `false`	yes = 1, no = 0
`scale`	a number on the grid `min`..`max` step `step` (≤ 101 points)	(value − min) / (max − min)
`options`	the `id` of one of 2–12 options, each with points	option points / best option points

`na_allowed` + `na_rule` say when a criterion may be skipped; `severe_error_enabled` + `severe_error_rule` say what counts as a severe error. The checklist-wide `scoring` block sets `max_score` (default 100), `rounding` (mode `half_up`, `half_even`, `floor`, `ceil` or `none`; 0–4 digits) and `severe_error_policy` (`flag_only` or `zero_total`).

The development sample checklist

`POST /cqa/checklists/dev-sample` creates a placeholder checklist that exercises every feature. It is labelled unofficial and must be replaced by the official checklist once that is supplied. All CQA examples below use it:

id	Section	Type	Weight	N/A	Severe
`greeting`	opening	binary	10	yes	—
`introduction`	opening	binary	5	yes	—
`need_identified`	resolution	scale 0–5	20	—	—
`solution`	resolution	options full=2 · partial=1 · none=0	30	—	yes
`tone`	resolution	scale 1–5	15	—	—

id	Section	Type	Weight	N/A	Severe
`further_help`	closing	binary	10	yes	—
`farewell`	closing	binary	10	yes	—

Scoring: max 100, `half_up` to 1 digit, `flag_only`, N/A weight redistributed.

Immutable versions

Any change to criteria, sections or scoring rules saves a **new version**; old versions are never changed. Every evaluation stores a full copy of the version it used, so past results can always be reproduced. Name, description and the archive flag are edited in place. Rules:

- an edit must say which version it was based on (`base_version`); if someone saved a newer version meanwhile, the API answers `409 stale_checklist` instead of silently overwriting their work;
- saving an identical config (same SHA-256 hash) creates no new version;
- `…/criteria/{id}/replace` swaps a criterion for one with a **new** id and records the replacement in the version history;
- a checklist used by any evaluation cannot be deleted (`409 checklist_in_use`); archive it instead;
- export produces `{"format": "atlas-cqa-checklist/1", …}`, which `POST /cqa/checklists/import` accepts.

Switching the severe-error policy creates version 2:

PUT /api/v1/cqa/checklists/{id}

```
{
  "base_version": 1,
  "config": "{…full config with severe_error_policy: \"zero_total\"…}",
  "note": "Severe errors zero the total"
}
```

← 200 OK

```
{
  "latest_version": 2,
  "versions": [
    {
      "version": 1,
      "created_at": "2026-11-07T13:30:22.779821+00:00",
      "note": "Development sample (unofficial placeholder criteria)",
      "config_hash": "dc59fc43a5d1ce1929b0e969f918b001bafffff6605ab3b7615623c4110736a81",
      "replacements": []
    },
    {
      "version": 2,
      "created_at": "2026-11-07T13:30:22.781474+00:00",
      "note": "Severe errors zero the total",
      "config_hash": "77354a8243aed0eee1cc3bfd3ad04d986b1ddc78b9fdf261ad4724604115ce42",
      "replacements": []
    }
  ]
}
```

A second edit still based on version 1 is refused:

PUT /api/v1/cqa/checklists/{id}

```
{"base_version": 1, "config": "{…}"}
```

← 409 Conflict

```
{
  "error": {
    "code": "stale_checklist",
    "message": "This checklist changed since you opened it (now v2, you edited v1). Reload it and apply
        your edits again.",
    "details": {"latest_version": 2}
  }
}
```

3.5 Running evaluations

`POST /cqa/evaluations` queues work and returns immediately with `202 Accepted`. Scope it with `conversation_ids` or a `group_id` (or neither, meaning every conversation). `only_missing: true` skips conversations that already have a queued or completed evaluation with the same checklist version. `adapter` picks the evaluator (default `ATLAS_CQA_EVALUATOR`), and the request is refused up front if that evaluator is not ready.

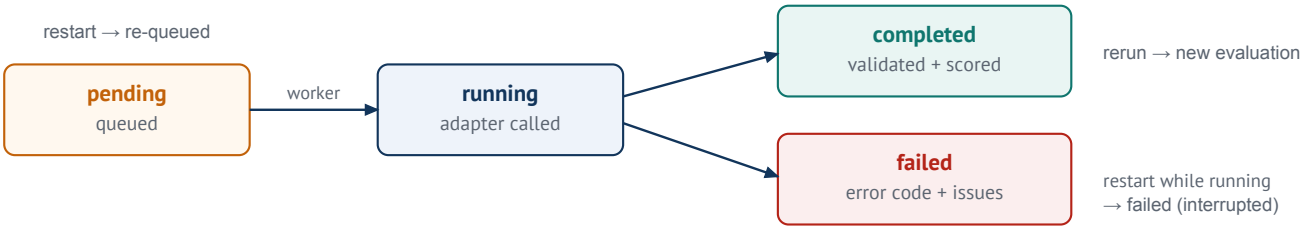


Figure 3. Evaluation states.

Queueing **snapshots** the checklist version and the conversation's transcript hash. A worker thread then:

- marks the evaluation `running`;
- fails it if the conversation is gone or its transcript changed since queueing;
- builds the contract request and calls the adapter;
- validates the output strictly (next section). Any violation fails the evaluation with every problem listed and the raw output kept for debugging; nothing partial is saved;
- scores the validated answers and saves `original_results`, `original_score` and `final_score` (identical until an appeal changes it).

Failure codes: `invalid_output`, `evaluator_crashed`, `evaluator_timeout`, `evaluator_unreachable`, `evaluator_http_error`, `evaluator_invalid_json`, `transcript_changed`, `conversation_missing`, `unknown_evaluator`, `interrupted`. After a restart, `pending` evaluations are re-queued and `running` ones are failed as `interrupted`. `POST …/{id}/rerun` queues a fresh evaluation with the same checklist version and evaluator; the old record stays.

Evaluate the whole fixture group with checklist v1 (one of six queued items shown):

POST /api/v1/cqa/evaluations

```
{"checklist_id": "<checklist id>", "checklist_version": 1, "group_id": "FIXTURE"}
```

← 202 Accepted


```
{
  "batch_id": "186b91911af24df1a5c7ed820b70cdda",
  "queued": 6,
  "skipped": 0,
  "evaluations": [
    {
      "id": "53995338c6d34fac9739232fc5ac4389",
      "conversation_id": "fx-en-001",
      "status": "pending",
      "checklist_version": 1,
      "evaluator": "Development mock (not AI)",
      "is_mock": true,
      "final_total": null,
      "stale": false
    },
    "… 5 more"
  ]
}
```

Polling the batch until done is true:

GET /api/v1/cqa/evaluations/batches/{batch_id}

← 200 OK


```
{
  "batch_id": "186b91911af24df1a5c7ed820b70cdda",
  "total": 6,
  "pending": 0,
  "running": 0,
  "completed": 6,
  "failed": 0,
  "done": true,
  "evaluation_ids": ["53995338c6d34fac9739232fc5ac4389", "4e00194aadfa409b8a3fc5d8fbffbf14", "…"]
}
```


Asking for the HTTP evaluator before its URL is configured:

POST /api/v1/cqa/evaluations


```
{"checklist_id": "…", "conversation_ids": ["fx-ru-001"], "adapter": "http"}
```


← 422 Unprocessable Content


```
{
  "error": {
    "code": "evaluator_unavailable",
    "message": "Evaluator 'Model service (HTTP)' cannot run: ATLAS_CQA_EVALUATOR_URL is not set.",
    "details": null
  }
}
```


3.6 The evaluator contract (cqa-eval/1)

The contract is the single seam between the backend and any AI model. The model receives an **EvaluationRequest** and must return a **ModelOutput**. `GET /cqa/contract` serves both JSON schemas with a worked example, and `GET /cqa/evaluations/{id}/request` shows the exact request a given evaluation sent, which helps when debugging a model.

What the model receives

GET /api/v1/cqa/evaluations/{evaluation_id}/request

← 200 OK


```
{
  "contract_version": "cqa-eval/1",
  "evaluation_id": "46d981f4c4354ec2852b0252447f9483",
  "conversation": {
    "conversation_id": "fx-ru-001",
    "language": "RU",
    "channel": "chat",
    "channel_raw": "agent chat",
    "audio_url": null,
    "messages": [
      {
        "id": "m1",
        "index": 0,
        "speaker": "client",
        "timestamp": "00:00",
        "offset_seconds": 0.0,
        "text": "Здравствуйте, как вывести деньги со счёта на карту?"
      },
      {
        "id": "m2",
        "index": 1,
        "speaker": "bot",
        "timestamp": "00:02",
        "offset_seconds": 2.0,
```

```
        "text": "Здравствуйте! Я виртуальный помощник. Подскажу по выводу средств или соединю с
            оператором."
      },
      "… m3 to m10"
    ]
  },
  "checklist": {
    "checklist_id": "c1748e9adb744ffb8d579cacf2af4ecd",
    "version": 1,
    "name": "Development sample checklist (unofficial)",
    "sections": [
      {"id": "opening", "name": "Opening", "description": "Placeholder section."},
      {"id": "resolution", "name": "Resolution", "description": "Placeholder section."},
      {"id": "closing", "name": "Closing", "description": "Placeholder section."}
    ],
    "criteria": [
      {
        "id": "greeting",
        "name": "Greeting (sample)",
        "description": "Sample criterion: the operator greets the client in their first message.",
        "section_id": "opening",
        "weight": 10.0,
        "answer_type": "binary",
        "scale": null,
        "options": null,
        "na_allowed": true,
        "na_rule": "Sample rule: no operator joined the conversation.",
        "severe_error_enabled": false,
        "severe_error_rule": ""
      },
      "… 6 more criteria"
    ],
    "scoring": {
      "max_score": 100.0,
      "rounding": {"mode": "half_up", "digits": 1},
      "severe_error_policy": "flag_only",
      "na_policy": "redistribute"
    }
  },
  "rules": {
    "evaluated_speaker": "operator",
    "context_speakers": ["client", "bot", "system"],
    "answer_formats": "…",
    "evidence": "…",
    "not_applicable": "…",
    "severe_error": "…"
  }
}
```


The `rules` block spells out the instructions in plain words: only the **operator** is evaluated (client, bot and system lines are context), how each answer type is expressed, and the evidence, N/A and severe-error rules below.

What the model must return

ModelOutput for fx-ru-001 (from app/cqa/examples.py)


```
{
  "contract_version": "cqa-eval/1",
  "evaluation_id": "0123456789abcdef0123456789abcdef",
  "conversation_id": "fx-ru-001",
  "checklist_id": "fedcba9876543210fedcba9876543210",
  "checklist_version": 1,
  "model": {"name": "example-model", "version": "0.0"},
  "results": [
    {
      "criterion_id": "greeting",
      "value": true,
      "raw_score": 1,
      "is_na": false,
      "explanation": "The operator's first message opens with a greeting.",
      "evidence": [{"message_id": "m5", "quote": "Добрый день!"}]
    },
    {
      "criterion_id": "introduction",
```

```
      "value": true,
      "is_na": false,
      "explanation": "The operator gives their name in the first message.",
      "evidence": [{"message_id": "m5", "quote": "Меня зовут Мария."}]
    },
    {
      "criterion_id": "need_identified",
      "value": 4,
      "is_na": false,
      "explanation": "The request (withdrawal to a card) is clear from the client's first line; the
          operator acts on it without asking clarifying questions.",
      "evidence": [
        {"message_id": "m1", "quote": "как вывести деньги со счёта на карту?"},
        {"message_id": "m5", "quote": "Помогу с выводом средств."}
      ]
    },
    {
      "criterion_id": "solution",
      "value": "full",
      "is_na": false,
      "explanation": "The operator gives the exact steps and the expected timing; the client confirms
          success.",
      "evidence": [{"message_id": "m6"}, {"message_id": "m7", "quote": "Получилось, заявка создана."}]
    },
    "… tone, further_help, farewell"
  ]
}
```


How the output is checked

Validation is strict and lists **every** problem, not just the first:

- **Schema.** Unknown fields, wrong types and coerced values are rejected (`"true"` is not `true`).
- **Identity.** `contract_version`, `evaluation_id`, `conversation_id`, `checklist_id` and `checklist_version` must match the request.
- **Coverage.** Every criterion is answered exactly once; unknown criterion IDs are errors.
- **Answers.** The value must fit the criterion: a boolean for binary, an on-grid number in range for a scale, a known option id for options.
- **Evidence.** Every non-N/A answer needs at least one evidence reference or an inspected span. Each `message_id` must exist. A `quote` must really occur in that message, compared ignoring case, extra spaces, surrounding quote marks and a trailing ellipsis; the matched character range is stored for highlighting.
- **N/A.** Only where `na_allowed`, with an `na_reason`, and `value` must be null.
- **Severe errors.** Only where enabled, with a reason, and never on an N/A answer.
- **Claimed scores.** A `raw_score` the model states is stored for comparison but never used; a mismatch only adds a warning.

Here the example output was changed in four ways (wrong quote, no evidence, out-of-range value, a missing criterion), and validation reports each one:

`ContractViolation.issues`

```
results[0] (greeting): evidence[0]: quote not found in m5.
results[1] (introduction): needs at least one evidence reference or inspected span.
results[2] (need_identified): 7 is outside the scale 0–5.
Missing results for: farewell.
```

`An extra field "confidence" fails schema validation`

```
results.3.confidence: Extra inputs are not permitted
```

Evidence without a quote is stored with the first 600 characters of the message (`quote_matched: false`); evidence with a quote stores the exact matched range. For example, the quote on `m7` matched characters 0–27: “Получилось, заявка создана.”.

3.7 Evaluator adapters

An adapter turns an `EvaluationRequest` into a `ModelOutput`. It never scores. Every module in `app/cqa/adapters/` is imported at start-up, so adding a file is enough; a broken adapter module is logged and skipped instead of taking the API down.

Adapter	What it does
`mock`	**Development only, not AI.** Answers come from a SHA-256 hash of (conversation, criterion, criterion definition), so they are repeatable and editing one criterion changes only that criterion. Evidence is a real transcript line quoted verbatim. Every explanation starts with `[DEV MOCK]`, results are flagged `is_mock`, and the overview counts them.
`http`	POSTs the request as JSON to `ATLAS_CQA_EVALUATOR_URL` and reads the ModelOutput back (8 MB limit, 180 s timeout). Only loopback hosts (`localhost`, `127.0.0.1`, `::1`, and `host.docker.internal` from inside Docker) are allowed unless `ATLAS_CQA_EVALUATOR_ALLOW_REMOTE=true`, so transcripts never leave the machine by accident.
your own	Subclass `EvaluatorAdapter`, register a name, return a dict or `ModelOutput`.

Adding the team's model as an in-process adapter

```
# apps/api/app/cqa/adapters/team_model.py
from app.cqa.adapters.base import EvaluatorAdapter, EvaluatorError, register_adapter
from app.cqa.contract import CONTRACT_VERSION, EvaluationRequest


@register_adapter("team-model")
class TeamModel(EvaluatorAdapter):
    label = "Team model"
    version = "0.1"

    def availability(self) -> tuple[bool, str]:
        return True, ""                      # or (False, "why it cannot run"), shown in the UI
    def evaluate(self, request: EvaluationRequest) -> dict:
        results = []
        for criterion in request.checklist.criteria:
            ...  # ask the model; append {"criterion_id", "value", "explanation", "evidence": [...]}
        return {"contract_version": CONTRACT_VERSION, "evaluation_id": request.evaluation_id,
                "conversation_id": request.conversation.conversation_id,
                "checklist_id": request.checklist.checklist_id,
                "checklist_version": request.checklist.version,
                "model": {"name": "team-model", "version": self.version}, "results": results}

# select it: ATLAS_CQA_EVALUATOR=team-model   (or "adapter": "team-model" per request)
```

Raising `EvaluatorError(message, code)` fails the evaluation with that code; any other exception is caught and recorded as `evaluator_crashed`.

3.8 The scoring engine

`compute_score(config, results)` is a pure function. It uses Python's `Fraction`, so 0.1 means exactly one tenth, not a binary float approximation, and rounding happens once, at the end, by the checklist's rule. The rules:

- **Normalize** each answer to 0–1 (see the answer-type table in 3.4).
- **Drop N/A** criteria and share their weight among the applicable ones: effective weight = weight / Σ applicable weights.
- **Points** = normalized × effective weight × max_score; **total** = Σ points, then rounded.
- **Severe errors** are listed separately. `flag_only` leaves the total unchanged; `zero_total` sets it to 0 (the pre-policy total is still reported).
- **Sections** report a subtotal on the same 0–max scale (weighted average of their applicable criteria) and their contribution in points.
- **Incomplete input** (missing answer, invalid answer, N/A where not allowed) gives status `incomplete` and no total. If every criterion is N/A the status is `not_applicable`, also with no total.

Example 1: the contract example output

Criterion	Answer	Normalized	Weight	Effective	Points
`greeting`	true	1	10	0.1	10.00
`introduction`	true	1	5	0.05	5.00
`need_identified`	4 on 0–5	0.8	20	0.2	16.00
`solution`	"full" (2 of 2)	1	30	0.3	30.00
`tone`	5 on 1–5	1	15	0.15	15.00
`further_help`	true	1	10	0.1	10.00
`farewell`	true	1	10	0.1	10.00
Total			100	applicable 100	**96**

Note the `tone` scale starts at 1, so 5 normalizes to (5 − 1) / (5 − 1) = 1 and a 1 would score 0.

Section results for the same answers: **Opening** 100 / 100 (contributes 15 of 15 points); **Resolution** 93.8 / 100 (contributes 61 of 65 points); **Closing** 100 / 100 (contributes 20 of 20 points).

Example 2: N/A redistributes weight

If the bot handled the opening and `greeting` and `introduction` are answered N/A, their 15 weight points drop out. The remaining 85 are scaled up, so `solution` now carries 30 / 85 = 0.3529 of the total:

Criterion	Answer	Normalized	Weight	Effective	Points
`greeting`	N/A	N/A	10	0	—
`introduction`	N/A	N/A	5	0	—
`need_identified`	4 on 0–5	0.8	20	0.2353	18.82
`solution`	"full" (2 of 2)	1	30	0.3529	35.29
`tone`	5 on 1–5	1	15	0.1765	17.65
`further_help`	true	1	10	0.1176	11.76
`farewell`	true	1	10	0.1176	11.76
Total			100	applicable 85	**95.3**

The exact total is 95.294118…, rounded half-up to one digit: **95.3**.

Example 3: a severe error

Change `solution` to `"none"` with a severe error (“wrong withdrawal instructions”). Under `flag_only` the total is **66** and `severe_errors` is `["solution"]`. Under `zero_total` the total becomes **0**, `severe_error_applied` is true, and `total_before_severe_policy` still shows 66.

Example 4: rounding

The mock evaluation of `fx-ru-001` (next section) sums to exactly 66.25. With `half_up` and one digit it becomes **66.3**; `half_even` would give 66.2, `floor` 66.2, `ceil` 66.3. Because the sum is an exact fraction, the result never depends on float noise (the code's own example: 84.45 → 84.5).

3.9 Appeals and reviewer overrides

An operator who disagrees with one criterion files an **appeal** with a comment. A reviewer then **confirms** the result or **overrides** it with a new answer and a mandatory explanation. An override is always a new answer, never a typed-in score: the engine recomputes the final score from the original answers plus the overrides. Original answers, the disputed result, the decision and the totals before and after are all kept, and every step is written to the audit trail.

- Only a **completed** evaluation can be appealed (`409 evaluation_not_completed`).
- One pending appeal per criterion per evaluation (`409 appeal_exists`).
- A decided appeal cannot be decided again (`409 appeal_already_decided`).
- The override is validated like an evaluator answer: N/A only where allowed, severe error only where enabled, and the value must fit the criterion (`422 invalid_override`). The override keeps the original evidence.
- `GET /cqa/appeals/{id}` returns the transcript lines around the evidence (±2 messages, up to 40) so the reviewer can decide without opening the whole conversation.

Worked example: fixing a wrong “solution” answer

The development mock evaluated `fx-ru-001` like this. Its `solution` answer is clearly wrong: the operator gave correct withdrawal steps in m6 and the client confirmed success in m7.

Criterion	Mock answer	Points	Severe
greeting	true	10 / 10	
introduction	true	5 / 5	
need_identified	5	20 / 20	
solution	"none"	0 / 30	yes
tone	4	11.25 / 15	
further_help	true	10 / 10	
farewell	true	10 / 10	
Total		**66.3** (exact 66.25)	

POST /api/v1/cqa/appeals

```
{
  "evaluation_id": "<evaluation id>",
  "criterion_id": "solution",
  "comment": "The steps in m6 worked: the client confirms in m7 that the request was created.",
  "submitted_by": "Operator Maria"
}
```

← 201 Created

```
{
  "id": "9d22170818cc40a9906b2844dc6a35bd",
  "status": "pending",
  "criterion_id": "solution",
  "disputed_result": {
    "criterion_id": "solution",
    "value": "none",
    "severe_error": true,
    "…": "explanation, evidence, spans"
  },
  "disputed_points": 0.0,
  "disputed_max_points": 30.0,
  "total_at_submission": 66.3
}
```

POST /api/v1/cqa/appeals/{appeal_id}/decision

```
{
  "decision": "override",
  "explanation": "Instructions in m6 are correct and the client confirms success in m7. Fully resolved;
      no severe error.",
  "reviewer": "QA lead",
  "override": {"value": "full", "severe_error": false}
}
```

← 200 OK

```
{
  "status": "overridden",
  "decision": {
    "decision": "override",
    "reviewer": "QA lead",
    "previous_result": {"value": "none", "severe_error": true},
    "new_result": {"value": "full", "severe_error": false, "source": "reviewer"},
    "total_before": 66.3,
    "total_after": 96.3
  }
}
```

The final score moved from **66.3** to **96.3** (30 points for a full solution, and the severe-error flag is gone). The original score is still stored next to it.

The audit trail, newest first:

GET /api/v1/cqa/audit?limit=3

← 200 OK

```
[
  {
    "at": "2026-11-07T13:30:22.854754+00:00",
    "type": "evaluation.final_recalculated",
    "actor": "scoring engine",
    "summary": "Final score of fx-ru-001 recalculated: 66.3 → 96.3."
  },
  {
    "at": "2026-11-07T13:30:22.854701+00:00",
    "type": "appeal.decided",
    "actor": "QA lead",
    "summary": "QA lead overrode 'Correct solution (sample)' on fx-ru-001."
  },
  {
    "at": "2026-11-07T13:30:22.851608+00:00",
    "type": "appeal.submitted",
    "actor": "Operator Maria",
    "summary": "Operator Maria appealed 'Correct solution (sample)' on fx-ru-001."
  }
]
```

3.10 Overview and audit

`GET /cqa/overview` is the dashboard in one call: conversation counts by language, channel, group and source; evaluations by status; mean final score and a 10-bin histogram; severe-error conversations; appeals pending and decided; overridden criteria; per-criterion statistics (scored, N/A, severe, mean normalized); how many latest results come from the mock; recent imports and activity. It focuses on one checklist: the one you pass, or else the checklist of the most recently completed evaluation.

After evaluating the six fixture conversations and deciding one appeal:

GET /api/v1/cqa/overview

← 200 OK


```
{
  "conversations": 6,
  "by_language": [
    {"value": "EN", "count": 2},
    {"value": "KZ", "count": 2},
    {"value": "RU", "count": 2}
  ],
  "by_channel": [{"value": "chat", "count": 5}, {"value": "voice", "count": 1}],
  "by_group": [{"value": "FIXTURE", "count": 6}],
  "by_source": [{"value": "fixture", "count": 6}],
  "evaluated_conversations": 6,
  "evaluations_by_status": [{"value": "completed", "count": 6}],
  "mean_final_score": 81.46666666666667,
  "max_score": 100.0,
  "score_bins": [
    {"start": 60.0, "end": 70.0, "count": 1},
    {"start": 70.0, "end": 80.0, "count": 1},
    {"start": 80.0, "end": 90.0, "count": 2},
    {"start": 90.0, "end": 100.0, "count": 2},
    "… empty bins omitted"
  ],
  "severe_error_conversations": 0,
  "appeals_pending": 0,
  "appeals_decided": 1,
  "overridden_criteria": 1,
  "checklist_id": "c1748e9adb744ffb8d579cacf2af4ecd",
  "checklist_name": "Development sample checklist (unofficial)",
  "checklist_version": 2,
  "criterion_stats": [
    {
      "criterion_id": "greeting",
      "name": "Greeting (sample)",
      "evaluated": 6,
      "scored": 4,
      "na": 2,
      "severe": 0,
      "mean_normalized": 1.0
    },
    {
      "criterion_id": "introduction",
      "name": "Introduces themselves (sample)",
      "evaluated": 6,
      "scored": 4,
      "na": 2,
      "severe": 0,
      "mean_normalized": 0.75
    },
    {
      "criterion_id": "need_identified",
      "name": "Identifies the need (sample)",
      "evaluated": 6,
      "scored": 6,
      "na": 0,
      "severe": 0,
      "mean_normalized": 0.7666666666666666
    }
  ],
  "mock_results": 6,
  "recent_imports": ["…"],
  "recent_activity": ["…"]
}
```


`GET /cqa/audit` filters the append-only trail by conversation, evaluation, checklist or appeal. Event types: `conversations.imported`, `conversations.import_failed`, `checklist.created`, `checklist.imported`, `checklist.version_saved`, `checklist.updated`, `checklist.criterion_replaced`, `checklist.duplicated`, `checklist.deleted`, `evaluation.queued`, `evaluation.completed`, `evaluation.failed`, `appeal.submitted`, `appeal.decided`, `evaluation.final_recalculated`.

4. Data Studio

The Data Studio is a general analysis workbench. Every computation runs on the server and returns bounded results (pages, aggregates, sampled points), so the browser never receives a whole dataset. Every result carries a **source line** (`meta`): dataset, rows used out of total, filters, sampling and aggregation, so a chart or number can always be traced back.

4.1 Datasets: upload, import, replace

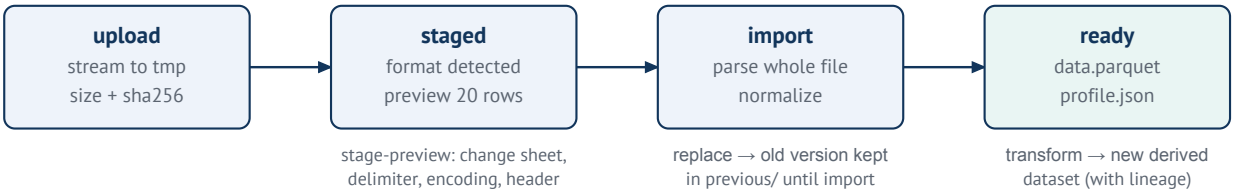


Figure 4. Dataset lifecycle.

- **Upload** (`POST /datasets`) streams the file to disk in 1 MB chunks, enforcing the size limit without holding it in memory, and records its SHA-256. The original is kept untouched.
- **Detect.** The extension must be `.csv`/`.tsv`/`.txt`, `.xlsx` or `.parquet`/`.pq`, and the content must match it (zip magic bytes for xlsx, `PAR1` for Parquet; a binary file named `.csv` is refused). For CSV: BOM, then UTF-8, then the best of cp1251, koi8-r, utf-16, cp1252, latin-1 (Cyrillic-friendly); delimiter sniffed among `,` `;` tab `|`. For Excel: the sheet list.
- **Preflight** refuses files that would exhaust memory using only metadata: an xlsx that expands past 4 GB (zip bomb) or a Parquet footer that declares too many rows or columns.
- **Stage preview** shows the first 20 rows; `POST …/stage-preview` re-parses with another sheet, delimiter, encoding, decimal mark or header row, and warns when only one column appears or encoding detection is unsure.
- **Import** parses the whole file (stopping one row past the limit), makes column names unique and non-empty (`column_3`, `amount_1`), converts text columns that are ⩾ 98% ISO dates, writes `data.parquet`, and profiles it. Each automatic conversion is reported in `notes`.
- **Replace** moves the current version into `previous/` until the new file is imported; `replace/cancel` restores it exactly.
- **Download** the processed table as CSV (streamed, formula-safe) or Parquet, or the original file.

Loading a bundled synthetic sample (stage + import in one call):

```
POST /api/v1/datasets/samples/customers_churn.csv

← 201 Created
```


```
{
  "id": "ace67d8352cc48438d3d10299f2c7426",
  "name": "customers_churn (synthetic)",
  "status": "ready",
  "source": "sample",
  "format": "csv",
  "n_rows": 6000,
  "n_cols": 13,
  "notes": ["'signup_date' parsed as datetime (ISO 8601)."],
  "columns": [
    {"name": "customer_id", "dtype": "str", "kind": "text"},
    {"name": "signup_date", "dtype": "datetime64[us]", "kind": "datetime"},
    {"name": "city", "dtype": "str", "kind": "categorical"},
    {"name": "segment", "dtype": "str", "kind": "categorical"},
    "…"
  ]
}
```

4.2 Profiling

Import builds a profile, cached in `profile.json`. For every column: type, kind (numeric, categorical, boolean, datetime, text), missing and unique counts, examples; numeric stats with quartiles, skew, IQR outlier fences and a histogram; top-10 categories; date range. For the dataset: duplicate rows, missing cells, and a Pearson correlation matrix for up to 30 numeric columns, computed on a disclosed random sample above 200,000 rows (all counts still use every row).

Quality flags state their **basis**: `detected` for measured facts, `heuristic` for guesses the team must verify. Unusual values are never called errors.

Flag	Basis	Raised when
`all_missing`, `high_missing`, `missing`	detected	no values / ⩾ 50% missing / any missing
`constant`	detected	a single distinct value
`duplicate_rows`	detected	exact duplicate rows exist
`high_cardinality`	detected	⩾ 90% distinct and more than 50 values
`potential_outliers`	heuristic	values outside 1.5 × IQR
`low_cardinality_numeric`	heuristic	2–10 distinct integers (maybe a category code)
`potential_date`, `numeric_as_text`	heuristic	text that is ⩾ 90% dates / ⩾ 95% numbers
`inconsistent_spelling`	heuristic	values differing only by case or spaces (`North` vs `north `)
`suspected_identifier`	heuristic	unique per row and looks like an ID (exclude from models)

Profile of customers_churn (excerpt):

GET /api/v1/datasets/{id}/profile

← **200 OK**


```
{
  "overview": {
    "n_rows": 6000,
    "n_cols": 13,
    "file_size_bytes": 426456,
    "memory_bytes": 703028,
    "duplicate_rows": 0,
    "duplicate_pct": 0.0,
    "missing_cells": 356,
    "missing_pct": 0.46,
    "kinds": {"text": 1, "datetime": 1, "categorical": 2, "numeric": 8, "boolean": 1}
  },
  "quality": [
    {
      "code": "suspected_identifier",
      "severity": "warning",
      "basis": "heuristic",
      "message": "'customer_id' is unique per row and looks like an identifier. Exclude it from models."
    },
    {
      "code": "high_cardinality",
      "severity": "info",
      "basis": "detected",
      "message": "'customer_id' has 6,000 distinct values (100% of rows)."
    },
    {
      "code": "missing",
      "severity": "info",
      "basis": "detected",
      "message": "'age' has 237 missing values (4.0%)."
    },
    {
      "code": "missing",
      "severity": "info",
      "basis": "detected",
      "message": "'balance_kzt' has 119 missing values (2.0%)."
```

```
    },
    "…"
  ]
}
```

4.3 Query, aggregate, export

All three share one filter engine (`services/filters.py`). A filter is `{column, op, value}` with ops `eq`, `neq`, `in`, `not_in`, `gt`, `gte`, `lt`, `lte`, `between`, `contains`, `not_contains`, `is_null`, `not_null`; values are coerced to the column's type, and a bad value gives a clear 422. `query` adds free-text search, up to 5 sort keys, column selection and paging (≤ 500 rows). `aggregate` groups by 1–3 columns with up to 10 metrics (`count`, `sum`, `mean`, `median`, `min`, `max`, `nunique`, `std`) and returns at most 1,000 groups. `export` streams every matching row as CSV.

Customers in Almaty aged 60+, richest first:

POST /api/v1/datasets/{id}/query

```
{
  "filters": [
    {"column": "city", "op": "eq", "value": "Almaty"},
    {"column": "age", "op": "gte", "value": 60}
  ],
  "sort": [{"column": "balance_kzt", "desc": true}],
  "columns": ["customer_id", "age", "balance_kzt", "churned"],
  "limit": 3
}
```

← 200 OK

```
{
  "columns": [
    {"name": "customer_id", "dtype": "str", "kind": "categorical"},
    {"name": "age", "dtype": "float64", "kind": "numeric"},
    {"name": "balance_kzt", "dtype": "float64", "kind": "numeric"},
    {"name": "churned", "dtype": "int64", "kind": "numeric"}
  ],
  "rows": [
    ["C102817", 74.0, 2183200.0, 0],
    ["C104492", 60.0, 946700.0, 0],
    ["C100236", 61.0, 898100.0, 0]
  ],
  "total_rows": 6000,
  "filtered_rows": 50,
  "offset": 0,
  "limit": 3
}
```

Churn rate and median balance per segment:

POST /api/v1/datasets/{id}/aggregate

```
{
  "group_by": ["segment"],
  "metrics": [
    {"agg": "count"},
    {"column": "churned", "agg": "mean"},
    {"column": "balance_kzt", "agg": "median"}
  ]
}
```

← 200 OK

```
{
  "columns": ["segment", "count", "mean_churned", "median_balance_kzt"],
  "rows": [
    ["retail", 4199, 0.1517027864, 226200.0],
    ["premium", 1199, 0.1217681401, 229750.0],
    ["business", 602, 0.1428571429, 219000.0]
  ],
  "total_groups": 3,
  "truncated": false,
  "meta": {
    "dataset_id": "ace67d8352cc48438d3d10299f2c7426",
    "dataset_name": "customers_churn (synthetic)",
```

```
    "n_rows_total": 6000,
    "n_rows_used": 6000,
    "filters": [],
    "sampled": false,
    "sample_size": null,
    "aggregation": "count(rows), mean(churned), median(balance_kzt) by segment",
    "notes": []
  }
}
```


4.4 Charts

`POST /datasets/{id}/visualize` builds chart-ready data on the server: `bar`, `line`, `scatter`, `histogram`, `box` and `heatmap`. Data is aggregated, binned or sampled so a chart never exceeds 5,000 points, 30 categories (the rest become “Other”) or 8 series. Dates on a line chart are bucketed by day, week, month, quarter or year (`auto` picks one). Invalid combinations, such as a mean of a text column, return a message the UI shows as is.

POST /api/v1/datasets/{id}/visualize


```
{"type": "bar", "x": "segment", "y": "churned", "agg": "mean"}
```


← 200 OK


```
{
  "x_label": "segment",
  "y_label": "mean of churned",
  "x_kind": "categorical",
  "series": ["value"],
  "rows": [
    {"x": "retail", "value": 0.1517027864},
    {"x": "business", "value": 0.1428571429},
    {"x": "premium", "value": 0.1217681401}
  ],
  "box": null,
  "heatmap": null,
  "meta": {
    "dataset_id": "ace67d8352cc48438d3d10299f2c7426",
    "dataset_name": "customers_churn (synthetic)",
    "n_rows_total": 6000,
    "n_rows_used": 6000,
    "filters": [],
    "sampled": false,
    "sample_size": null,
    "aggregation": "mean of churned by segment",
    "notes": []
  },
  "warnings": []
}
```


4.5 Preparation recipes (transforms)

`POST /datasets/{id}/transform` applies an ordered recipe to a **copy** of the dataset. With `preview: true` it returns the result without saving; otherwise it creates a new derived dataset whose `lineage` records the parent, the steps, the step log and the split. The source is never modified.

Group	Operations
Columns	`drop_columns`, `select_columns`, `rename`
Rows	`drop_duplicates`, `drop_missing`, `filter_rows`
Clean	`fill_missing` (mean, median, mode, constant), `parse_dates`, `cast` (numeric, string, category, datetime, boolean)
Derive	`derive_date_parts`, `derive_ratio`, `derive_log`, `bin_numeric` (quantile bins)
Encode / scale	`one_hot` (⩽ 50 categories), `scale` (standard or min-max)

Fitted steps (fill with mean/median/mode, one-hot, scale, quantile bins) log the exact values they learned. If a `split` is configured (`random`, `stratified` on `stratify_column`, or `chronological` on `time_column`), a `split` column

marks train and validation rows, and fitted steps **learn from training rows only**, so validation data never leaks into the preparation.

The age median (38) was fitted on the 4,800 training rows only:

POST /api/v1/datasets/{id}/transform

```
{
  "steps": [
    {"op": "drop_columns", "columns": ["customer_id"]},
    {"op": "fill_missing", "columns": ["age"], "strategy": "median"},
    {"op": "derive_date_parts", "columns": ["signup_date"]}
  ],
  "split": {"strategy": "stratified", "stratify_column": "churned", "test_size": 0.2},
  "preview": true
}
```

← 200 OK


```
{
  "preview": true,
  "n_rows_before": 6000,
  "n_cols_before": 13,
  "n_cols_after": 18,
  "log": [
    {
      "op": "split",
      "description": "Assigned 'split': 4,800 train / 1,200 validation (stratified split, seed 42).",
      "fitted": {},
      "fit_scope": "none"
    },
    {
      "op": "drop_columns",
      "description": "Dropped 1 column(s): customer_id.",
      "fitted": {},
      "fit_scope": "none"
    },
    {
      "op": "fill_missing",
      "description": "Filled 237 missing value(s) in 1 column(s) using median.",
      "fitted": {"age": "38.0"},
      "fit_scope": "train_only"
    },
    {
      "op": "derive_date_parts",
      "description": "Derived 5 date-part column(s).",
      "fitted": {},
      "fit_scope": "none"
    }
  ]
}
```


4.6 Models and experiments

Modeling is a guarded baseline workflow, not AutoML. First `POST /experiments/check` validates the setup and looks for leakage; then `POST /experiments` queues training on a 2-thread pool and returns `202` straight away. Poll `GET /experiments/{id}` for the result.

Model	Tasks	Notes
`dummy`	both	Always trained alongside your model on the same split as the baseline.
`logistic_regression`	classification	Scaled numeric features, one-hot categoricals.
`linear_regression`, `ridge`	regression	
`random_forest`	both	Native feature importances.
`catboost`	both	Only if the optional package is installed (`/health` reports `catboost`).

Pre-training checks

The check suggests a task (classification for up to 15 integer levels or text targets), drops columns that cannot carry signal (constant, all-missing, free text, identifiers) and warns about hazards. It warns but never silently drops a feature you chose:

- **critical**: `leakage_correlation` (|r| ⩾ 0.98 with the target), `leakage_purity` (each value maps to one class, ⩾ 99.5% purity), `single_class`, `too_many_classes` (> 50), `too_few_rows` (< 20), `no_features`, `target_not_numeric`;
- **warning / info**: `identifier_feature`, `name_overlap` (feature name contains the target's), `class_imbalance`, `tiny_class`, `target_missing`, `datetime_feature` (expanded into parts), `upstream_fit_leakage` (a derived dataset whose fitted steps used all rows).

POST /api/v1/experiments/check

```
{"dataset_id": "<dataset id>", "target": "churned", "task": "classification"}
```

← 200 OK


```
{
  "target": "churned",
  "target_kind": "numeric",
  "suggested_task": "classification",
  "task": "classification",
  "n_rows": 6000,
  "n_target_missing": 0,
  "n_classes": 2,
  "class_balance": [{"value": "0", "count": 5131}, {"value": "1", "count": 869}],
  "dropped_features": [{"column": "customer_id", "reason": "looks like an identifier"}],
  "warnings": [
    {
      "code": "datetime_feature",
      "severity": "info",
      "column": "signup_date",
      "message": "'signup_date' is expanded into year/month/day/weekday features."
    }
  ],
  "can_train": true,
  "usable_features": [
    "signup_date",
    "city",
    "segment",
    "age",
    "tenure_months",
    "monthly_fee_kzt",
    "…"
  ]
}
```


Training

Rows with a missing target are dropped. The split is random, stratified or chronological (the last `test_size` share by time). Training rows above 200,000 are sampled. Dates become year/month/day/weekday features, and all preprocessing lives inside the scikit-learn pipeline, so it is fitted on training rows only. Metrics are computed on the held-out test rows only:

- classification: accuracy, balanced accuracy, precision, recall and F1 for the positive class (the minority class in training), macro and weighted F1, ROC AUC and PR AUC; a confusion matrix; primary metric ROC AUC (binary) or macro F1;
- regression: MAE, RMSE, R^2, median AE, MAPE (when no actual is 0); predicted vs actual points and a residual histogram; primary metric RMSE;
- top-30 feature importances, test-set predictions as CSV (`predictions.csv`) and a metrics file (`metrics.json`). Experiments interrupted by a restart are marked failed.

POST /api/v1/experiments

```
{
  "dataset_id": "<dataset id>",
  "target": "churned",
  "task": "classification",
  "model": "random_forest",
  "exclude": ["customer_id"],
  "split": {"strategy": "stratified", "test_size": 0.2, "random_state": 42}
}
```

← 202 Accepted

```
{
  "id": "cecee4e9fa6d406da6259ad8c25c6c09",
  "status": "queued",
  "name": "random_forest → churned",
  "model": "random_forest",
  "task": "classification"
}
```

GET /api/v1/experiments/{id} (after it completes)

← 200 OK

```
{
  "status": "completed",
  "primary_metric": {"name": "roc_auc", "value": 0.727429, "higher_is_better": true},
  "baseline_metric": {"name": "roc_auc", "value": 0.5, "higher_is_better": true},
  "metrics": {
    "accuracy": 0.860833,
    "balanced_accuracy": 0.527274,
    "precision": 0.769231,
    "recall": 0.057471,
    "f1": 0.106952,
    "roc_auc": 0.727429,
    "pr_auc": 0.342133
  },
  "baseline_metrics": {
    "accuracy": 0.855,
    "balanced_accuracy": 0.5,
    "precision": 0.0,
    "recall": 0.0,
    "f1": 0.0,
    "roc_auc": 0.5,
    "pr_auc": 0.145
  },
  "positive_label": "1",
  "split_info": {"strategy": "stratified", "n_train": 4800, "n_test": 1200},
  "feature_importance": [
    {"feature": "signup_date", "importance": 0.255234},
    {"feature": "days_since_login", "importance": 0.142893},
    {"feature": "balance_kzt", "importance": 0.107759},
    {"feature": "tenure_months", "importance": 0.101617},
    "…"
  ],
  "duration_s": 0.291
}
```

Note. How to read this: accuracy looks fine (0.861) but barely beats the dummy baseline (0.855), because only about 14.5% of customers churned. ROC AUC (0.727 vs 0.500) shows the model does rank churners higher, yet recall at the default threshold is only 0.057. This is exactly why every run is reported next to its baseline.

4.7 Analysis plug-ins

An analysis is a Python function registered with `@register_analysis`. It receives an `AnalysisContext` (the filtered table plus helpers that build tables, charts and the source line) and a validated Pydantic parameter object, and returns summary figures, a table, an optional chart and notes. `GET /analyses` lists every analysis with its parameter JSON schema; parameters declared with `column_param(kinds=[...])` render as column pickers, so a new analysis needs **no frontend change**.

id	What it does
`value_concentration`	Value concentration (Pareto)
`group_comparison`	Group comparison
`association_ranking`	Association ranking
`kmeans_segments`	K-means segmentation
`anomaly_scores`	Anomaly scoring (Isolation Forest)
`time_trend`	Time trend
`challenge_template_segment_kpis`	Template · KPIs by segment

The built-ins: Pareto concentration with Gini coefficient; group comparison with a Kruskal–Wallis test; association ranking by Spearman ρ, correlation ratio η or Cramér's V depending on column types; k-means segments on standardized features; Isolation Forest anomaly scores; and period trends with change and a 3-period rolling mean. Each states its limits in `notes` (for example, “unusual is not the same as wrong”).

Do balances differ by city? (No: p = 0.555.)

POST /api/v1/datasets/{id}/analyses/group_comparison

```
{"params": {"group": "city", "metric": "balance_kzt"}}
```

← 200 OK


```
{
  "summary": [
    {"label": "Groups compared", "value": "6", "hint": null},
    {"label": "Highest mean", "value": "461,805.92", "hint": "Shymkent"},
    {"label": "Lowest mean", "value": "372,999.78", "hint": "Pavlodar"},
    {"label": "Kruskal-Wallis p", "value": "0.5552", "hint": "ε² = 0.001"}
  ],
  "notes": [
    "A small p-value says the groups' distributions differ; it does not say why or by how much in
        business terms, and it does not establish causation."
  ],
  "table": {
    "columns": ["group", "count", "mean", "median", "std", "min", "max"],
    "rows": [
      ["Shymkent", 794, 461805.9194, 233850.0, 970941.9134, 8000.0, 19240300.0],
      ["Aktobe", 551, 432529.7641, 237800.0, 633766.7457, 3800.0, 6768900.0],
      ["Karaganda", 637, 414858.2418, 209700.0, 653852.9823, 7900.0, 9109100.0]
    ]
  }
}
```


Registering a case-specific analysis (see app/challenge/analyses.py)

```
from pydantic import BaseModel
from app.extensions.registry import AnalysisContext, column_param, register_analysis

class Params(BaseModel):
    segment: str = column_param(kinds=["categorical"], title="Segment column")
    metric: str = column_param(kinds=["numeric"], title="Metric column")
@register_analysis("my_kpis", category="challenge", title="KPIs by segment", params=Params)
def my_kpis(ctx: AnalysisContext, p: Params):
    table = ctx.frame.groupby(p.segment)[p.metric].agg(["count", "sum", "mean"]).reset_index()
    return ctx.result("my_kpis", f"{p.metric} by {p.segment}", table=ctx.table(table))
```

`app/challenge/` is the place for the hackathon case: analyses registered there with `category="challenge"` appear on the Challenge page automatically, and `challenge/router.py` can hold bespoke endpoints under

`/api/v1/challenge`. Today it contains one template analysis and `GET /challenge/info`.

4.8 Findings, pitch deck and saved views

`ReportStore` keeps three small JSON documents. A **finding** records a conclusion with its chart, dataset or experiment, methodology, evidence items, interpretation, implication and limitations; findings can be reordered and exported as Markdown or JSON. The **pitch deck** has seven default slides (title, problem, observations, solution, methodology, results, impact and limitations) that reference findings by ID. **Saved views** store an explorer or chart configuration per dataset.

POST /api/v1/findings

```
{
  "title": "Retail customers churn most",
  "description": "Mean churn by segment.",
  "dataset_id": "<dataset id>",
  "interpretation": "Retail churn is 15.2%, premium 12.2%.",
  "limitations": "Synthetic data."
}
```

← 201 Created

```
{
  "id": "2c4f1a76694f4189b20647d9131f76ec",
  "include_in_pitch": true,
  "created_at": "2026-11-07T13:30:23+00:00",
  "…": "the fields sent"
}
```

5. The web server (apps/web-server)

`server.mjs` is a dependency-free Node server. It serves the static frontend from `apps/web` and forwards `/api/*` to FastAPI, so the browser only ever talks to one origin (no CORS in production). It is also the public boundary: behind Cloudflare it decides which API paths the internet can reach.

Request	What happens
`GET /healthz`	`{"status":"ok","service":"web","release":"<sha>"}` (used by Docker).
`/api/...` with `%2f`, `%5c`, `\`, `..` or `.` segments	`400 bad_path`: never forwarded ambiguously.
`/api/...` outside `API_ALLOWED_PREFIXES`	`404 not_found` without reaching the API. Default allow-list: `/api/v1/health` and `/api/v1/cqa/`, so the Data Studio (uploads, training) is not public.
write method while `API_READ_ONLY=true`	`405 read_only` with `Allow: GET, HEAD, OPTIONS` (view-only demo).
any other `/api/...`	Proxied to `API_UPSTREAM`. Hop-by-hop headers are stripped, `X-Forwarded-*` added, and `Host` is kept unchanged so the API's host and origin guards still work.
everything else	Static file from `STATIC_ROOT`.

Timeouts: 60 s for request headers, 15 minutes per request, 10 minutes idle per upstream call (`API_TIMEOUT_MS`), and a 120 s keep-alive that is deliberately longer than cloudflared's 90 s. `server.test.mjs` covers this behaviour.

> **Watch out.** With the default allow-list every CQA endpoint is reachable through the web server, including imports, evaluations, checklist edits and appeal decisions, and there is no login. For a public demo, set `API_READ_ONLY=true` or put an access layer (for example Cloudflare Access) in front.

6. Running, deployment and tests

6.1 Local development

Commands

```
cd apps/api
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt -r requirements-dev.txt
.venv/bin/pip install -r requirements-optional.txt              # optional: CatBoost
.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000     # API, docs at :8000/docs
node ../web-server/server.mjs                                   # UI + proxy at :3000
.venv/bin/python -m pytest -q                                   # 190 passed
```

6.2 Docker and Compose

`Dockerfile.api` builds a two-stage Python 3.14-slim image: dependencies are installed with `requirements.lock` as constraints, the app runs as a non-root user (uid 10001), data lives in `/data`, and a health check calls `/api/v1/health`. Uvicorn runs **one worker** with proxy headers enabled. One worker is required: the CQA store and both thread pools live in process memory, so several processes would each hold their own diverging copy.

`compose.prod.yml` runs two services:

- **api**: not published to the host (only `expose: 8000`); the data folder is bind-mounted to `/data`; `host.docker.internal` points at the host so a model service running there counts as local for the `http` adapter; CQA settings come from the environment and `api.env`.
- **web**: published on `127.0.0.1:3000` only (Cloudflare Tunnel provides public access); starts after the API is healthy; `API_ALLOWED_PREFIXES` and `API_READ_ONLY` control exposure.

CI/CD workflows, the Windows auto-deploy scripts and Cloudflare setup are documented in `docs/CI_CD.md`, `docs/DEPLOYMENT.md`, `docs/WINDOWS_SETUP.md` and `docs/CLOUDFLARE_SETUP.md`; they are outside the scope of this backend report.

6.3 Tests

`apps/api/tests` has 15 modules: CQA ingestion, checklists, scoring, contract, workflow, HTTP adapter; datasets, transforms, profile and analysis, modeling, findings and extensions; security. All 190 tests passed on 7 November 2026 (8 seconds). The examples in `app/cqa/examples.py` are themselves tested to pass the contract validation.

6.4 Limits and things to know

- **Placeholders.** The mock evaluator and the development checklist only demonstrate the workflow. Do not present their scores as quality results; the overview reports how many results are mock.
- **No authentication.** Exposure is controlled only by the web server's allow-list and read-only switch.
- **Scale.** CQA keeps every conversation, evaluation and appeal in memory and rescans them for lists and the overview. That is fine for hackathon volumes (hundreds to thousands of conversations), not for millions.
- **Recordings expire.** Audio links are signed for about an hour; the API re-resolves them on demand.
- **Restarts.** Running evaluations and trainings are marked failed on restart; pending evaluations are re-queued.

Appendix: endpoint reference

All 74 endpoints, taken from the app's OpenAPI schema. Paths are under `/api/v1`. Through the public web server only `/health` and `/cqa/*` are reachable by default.

CQA (29)

Method	Path	What it does
GET	/cqa/info	Source configured?, fixture groups, evaluators, contract version.
GET	/cqa/overview	Dashboard statistics.
GET	/cqa/contract	Contract JSON schemas and a worked example.
GET	/cqa/audit	Audit events, filterable.
GET	/cqa/imports	Last 200 import runs.
POST	/cqa/imports	Import a group (organizers' API or fixture).
GET	/cqa/conversations	Filter, sort, page conversations with facets.
GET	/cqa/conversations/{conversation_id}	Transcript, warnings, evaluations.
GET	/cqa/conversations/{conversation_id}/audio	Resolve a playable recording URL.
GET	/cqa/checklists	Checklist summaries (include_archived).
POST	/cqa/checklists	Create a checklist (v1).
POST	/cqa/checklists/dev-sample	Create the unofficial sample checklist.
POST	/cqa/checklists/import	Import an exported checklist.
GET	/cqa/checklists/{checklist_id}	Checklist with all versions.
PUT	/cqa/checklists/{checklist_id}	Edit metadata and/or save a new version.
DELETE	/cqa/checklists/{checklist_id}	Delete if unused (else archive).
POST	/cqa/checklists/{checklist_id}/duplicate	Copy a version as a new checklist.
POST	/cqa/checklists/{checklist_id}/criteria/{criterion_id}/replace	Replace a criterion (new version).
GET	/cqa/checklists/{checklist_id}/export	Download a version as JSON.
POST	/cqa/evaluations	Queue evaluations (202).
GET	/cqa/evaluations	Evaluations, filterable and paged.
GET	/cqa/evaluations/batches/{batch_id}	Batch progress.
GET	/cqa/evaluations/{evaluation_id}	Per-criterion results, scores, appeals, audit.
GET	/cqa/evaluations/{evaluation_id}/request	The exact contract request sent.
POST	/cqa/evaluations/{evaluation_id}/rerun	Queue a fresh evaluation.
POST	/cqa/appeals	Appeal one criterion.
GET	/cqa/appeals	List appeals (status, evaluation, conversation).
GET	/cqa/appeals/{appeal_id}	Appeal with criterion, scores and transcript context.
POST	/cqa/appeals/{appeal_id}/decision	Confirm or override.

System (1)

Method	Path	What it does
GET	/health	Status, version, CatBoost availability, accepted formats, limits.

Datasets (15)

Method	Path	What it does
GET	/datasets	List datasets, newest first.
POST	/datasets	Upload a file (multipart) and stage it; returns a preview.
GET	/datasets/samples	List bundled synthetic sample files.
POST	/datasets/samples/{filename}	Stage and import a sample in one call.
GET	/datasets/{dataset_id}	Dataset manifest: status, columns, notes, lineage.
PATCH	/datasets/{dataset_id}	Rename.
DELETE	/datasets/{dataset_id}	Delete the dataset folder.
POST	/datasets/{dataset_id}/stage-preview	Re-parse a staged file with other options.
POST	/datasets/{dataset_id}/import	Parse the whole file, store Parquet, profile.
POST	/datasets/{dataset_id}/replace	Stage a new file under the same id (old kept).
POST	/datasets/{dataset_id}/replace/cancel	Restore the previous version.
GET	/datasets/{dataset_id}/preview	First rows (⩽ 500).
GET	/datasets/{dataset_id}/profile	Cached profile (`refresh=true` to rebuild).
POST	/datasets/{dataset_id}/profile/refresh	Rebuild the profile.
GET	/datasets/{dataset_id}/download	CSV, Parquet or the original file.

Analysis (5)

Method	Path	What it does
POST	/datasets/{dataset_id}/query	Filter, search, sort, page rows.
POST	/datasets/{dataset_id}/aggregate	Group-by with metrics.
POST	/datasets/{dataset_id}/visualize	Chart-ready aggregated data.
POST	/datasets/{dataset_id}/transform	Preview or save a preparation recipe.
POST	/datasets/{dataset_id}/export	All filtered rows as CSV.

Analysis plug-ins (2)

Method	Path	What it does
GET	/analyses	Registered analyses with parameter schemas.
POST	/datasets/{dataset_id}/analyses/{analysis_id}	Run an analysis.

Models and experiments (8)

Method	Path	What it does
GET	/models	Model options and availability.
POST	/experiments/check	Validate target and features; leakage warnings.
POST	/experiments	Queue training (202).
GET	/experiments	List experiments (optionally per dataset).
GET	/experiments/{experiment_id}	Status and full results.
DELETE	/experiments/{experiment_id}	Delete record and predictions.

Method	Path	What it does
GET	/experiments/{experiment_id}/predictions.csv	Test-set predictions.
GET	/experiments/{experiment_id}/metrics.json	Metrics file download.

Findings, pitch, views (13)

Method	Path	What it does
GET	/findings	List findings.
POST	/findings	Create a finding.
PUT	/findings/order	Reorder findings.
GET	/findings/export.md	Findings and pitch as Markdown.
GET	/findings/export.json	Findings and pitch as JSON.
GET	/findings/{finding_id}	One finding.
PATCH	/findings/{finding_id}	Edit a finding.
DELETE	/findings/{finding_id}	Delete a finding.
GET	/pitch	The pitch deck.
PUT	/pitch	Save the pitch deck.
GET	/views	Saved views (optionally per dataset).
POST	/views	Save a view.
DELETE	/views/{view_id}	Delete a view.

Challenge (1)

Method	Path	What it does
GET	/challenge/info	Case title and challenge analyses.

Glossary

Term	Meaning
Checklist version	An immutable snapshot of criteria, sections and scoring rules; evaluations pin one.
Criterion	One thing the operator is judged on, with a weight and an answer type.
Evaluator / adapter	The component that answers the criteria for a conversation (mock, http, your model).
Evidence	A message id (and optional verbatim quote) that supports an answer.
Inspected span	A range of messages checked when an expected action is missing.
N/A	Criterion not applicable; its weight is shared among the others.
Severe error	A flagged failure reported next to the score, or zeroing it under `zero_total`.
Original vs final score	The evaluator's result vs the result after reviewer overrides.
Stale evaluation	Its conversation's transcript changed after it ran.
Fixture	Synthetic, hand-written test data (`FIXTURE…` groups, `fx-…` ids), never organizer data.
Source line (`meta`)	Provenance attached to every Data Studio result: rows used, filters, sampling.